RAG Architecture for Personalized Zettel-Based Knowledge Graph Chatbots

Executive Summary

Personal-knowledge products such as Google NotebookLM, Obsidian AI plugins, Mem, Notion AI Q&A, Readwise Reader, and Perplexity Spaces all implement some form of retrieval-augmented generation (RAG) over user-curated content, typically combining vector search over chunked documents with an LLM that generates source-cited answers. Public documentation and credible commentary indicate that leading systems rely on embedding-based retrieval over document passages, often with workspace- or notebook-level scoping, and emphasize inline citations and provenance to increase trust. The most consistent user complaints concern hallucinations despite RAG, weak control over retrieval scope, limited graph-like sense-making, and constraints on cross-source or cross-project reasoning.[1][2] [3][4] [5][6] [7][8] [9][10] [11][12] [13][14] [15][16]

For a Zettel-based product with pre-computed summaries, tags, and a personal Knowledge Graph, the most practical high-quality architecture is a hybrid of dense and graph-augmented retrieval over per-user pgvector indexes inside Supabase, coupled with a strong embedding model, cross-encoder reranking, and a graph-aware context assembly layer that respects user selection of Zettels. This blueprint uses FastAPI as the orchestration layer, Supabase/PostgreSQL with pgvector for both relational and vector storage, a modern embedding model such as NV-Embed-v2 or BGE-large, BM25-style lexical retrieval via PostgreSQL full-text search, BGE-Reranker or similar cross-encoder reranking, and a major LLM (e.g., GPT-4-class or Gemini/Groq-equivalent) with carefully engineered prompts for grounded, citation-rich answers. Evaluation should rely on automated RAG frameworks such as RAGAS and TruLens plus targeted manual review, measuring retrieval metrics such as recall, MRR, and NDCG along with generation metrics like faithfulness and answer relevance.[17][18] [19][20] [21][22] [23][24] [25][26] [27][28] [29][30] [^31]

Competitive Landscape: Personal / Document-Level RAG Products

Google NotebookLM

NotebookLM is marketed as an AI research assistant grounded in user-provided documents; it lets users upload PDFs, Google Docs, slides, and other files, then ask questions, get summaries, and generate reports based only on these sources. Product materials and independent explainers state that NotebookLM splits sources into small passages, embeds them in a vector space, retrieves the most relevant chunks for a query, and has the LLM answer using only these chunks, with inline citations to the original passages, which is a standard RAG pattern. User commentary in PKM communities reports that NotebookLM excels at query-first workflows and quick recall over a pile of documents but lacks features

such as backlinks, graph-like structure, and long-term knowledge development tools, making it more of an intelligent archive than a full PKM.[32][7] [9][12] [13][33] [15][16]

NotebookLM's praised capabilities include grounded answers with clear citations and strong summarization and synthesis over multiple documents. Criticisms emphasize source limits per notebook, isolation between notebooks (no cross-notebook context), lack of visual connections, limited export and API support, and the absence of graph-like tooling, all of which constrain large, evolving personal knowledge bases. Architectural details beyond the high-level RAG design are not public; any assumptions about internal ranking algorithms, rerankers, or query expansion would be speculative and should be treated as such.[9][13] [33] [16] [^32]

Obsidian AI / RAG Plugins

Obsidian has several community plugins that enable RAG-style chat over local notes, including Mini-RAG and Vault AI Chat. Mini-RAG connects Obsidian notes to locally running LLMs via Ollama and supports starting a chat in the context of a note or folder so that retrieval is restricted to relevant content, enabling retrieval-augmented responses grounded in the vault. Vault AI Chat similarly provides an AI assistant inside the vault that "reads your notes for context" with RAG and supports multiple AI providers, streaming responses, and settings to control how many notes are used as context.[1][34] [^35]

Users appreciate that these plugins keep data local, integrate directly into their existing note-taking workflow, and can use any self-hosted or API-backed model. Limitations include community-maintained quality, configuration complexity, and limited large-scale indexing or evaluation support; detailed internal architectures are not fully documented, but high-level descriptions indicate basic vector search over file contents and simple ranking based on similarity scores. Compared to NotebookLM or Mem, Obsidian plugins typically lack cloud-scale multi-device knowledge graphs but offer stronger control over storage, models, and privacy, which some power users value.[34][1]

Mem.ai

Mem is positioned as an AI-powered notes app and workspace that automatically organizes notes and provides fast, AI-driven search and "Mem Chat" for conversational retrieval over personal and team knowledge. Product descriptions highlight instant AI-driven search, automatic tagging and linking of related notes, GPT-4 integration for summarization and Q&A, and Mem Chat for asking questions about your workspace, suggesting an internal search plus RAG-like chat over notes and context. Guides for Mem Chat emphasize personal retrieval use cases like "Where did I save that thing?" and state that the system can find items across typed, spoken, clipped, or forwarded notes, implying embedded representations and semantic search.[2][6] [^36]

User and reviewer commentary portrays Mem as a "second brain" that becomes powerful once a critical mass of notes is accumulated, with strengths in automatic linking and context-aware recall. Critiques note limitations in collaboration, structured views, integrations, and automation, and some reviews describe the workspace as "read-mostly" due to limited agentic workflows and external integrations. No detailed public description of Mem's RAG

architecture exists; it is reasonable but speculative to assume a standard vector-search-plus-LLM pipeline with some metadata and tag-based filtering.[37][38] [^2]

Notion AI Q&A

Notion's Q&A feature lets users ask natural-language questions and get answers based on the content available in their workspace, respecting page and database permissions. Commentary and product blogs describe the system as a "colossal RAG operation" over the domain's workspace that uses retrieval-augmented generation to mitigate hallucinations while enforcing identity and access checks so that users only see content they are allowed to access. External analysis points out that Notion Q&A cannot see inside certain embedded content (e.g., PDFs or Google Docs when embedded rather than imported) and that it tends to prefer recently edited or viewed pages, introducing recency bias into retrieval.[3][39] [40][41]

Users praise Q&A for quickly surfacing internal information such as project status, policies, and meeting notes directly in the Notion UI. Criticisms focus on hallucinations even when the answer exists in the workspace and on opaque retrieval choices, including recency bias and inability to search embedded third-party content, which can reduce reliability for foundational documents. Architectural details like embeddings, rerankers, and exact indexing are not fully disclosed; the use of vector search plus metadata filters and BM25-like lexical signals is plausible but cannot be stated as fact without explicit documentation.[39][8] [40][3]

Readwise Reader / Ghostreader

Readwise Reader is a read-it-later and document/highlight management tool whose Ghostreader feature uses GPT-based models to summarize documents, explain terms, translate, and "ask the document a question." Official docs explain that Ghostreader can be invoked on the whole document (via Shift+G) or on selected text, and that prompts include summarization, simplification, definitions, and custom prompts, all answered in the context of the document. Reviews and blog posts note that Ghostreader is built on OpenAI's GPT-3/3.5 models and that it tailors answers to the context of the highlighted text or document, effectively acting as a single-document RAG or context-injected LLM assistant.[4][42] [10][43] [44][45]

Users value Reader for its world-class reading and web clipping experience and consider Ghostreader a strong contextual assistant within documents, especially for summarization and question answering over a single file. Limitations include standard LLM issues such as hallucinations (acknowledged by Readwise and reviewers), reliance on a single-document scope (rather than cross-library retrieval), and dependence on OpenAI APIs, which may raise privacy or cost concerns. There is no public evidence of multi-document vector search or cross-user personalization; Ghostreader appears primarily as a per-document, prompt-engineering-based augmentation rather than a full multi-source RAG architecture.[42][10] [^45]

Perplexity Spaces and Internal Knowledge Search

Perplexity's Spaces feature lets users create dedicated workspaces where they upload files and configure custom instructions, then run threads that combine web search with their uploaded data. Help-center and knowledge documents explain that Internal Knowledge Search and Spaces allow Pro and Enterprise users to search across internal files alongside the web, with inline citations clearly indicating which pieces of an answer come from internal documents vs public web sources. External comparisons highlight that Spaces are powerful for research but mix internal uploads with web search, which can blur which parts of an answer are from private data vs the open web, and note that Spaces currently rely mainly on uploaded files rather than connectors to tools like Slack or GitHub.[5][46] [11][14] [^47]

Users appreciate Perplexity's fast, source-cited answers and the ability to scope queries to particular Spaces, effectively creating topic- or project-specific RAG environments. Critiques focus on the mixing of internal and web knowledge (potentially confusing in sensitive domains) and the lack of connectors to broader internal tools, which constrains it as an internal knowledge management platform. Perplexity's underlying architecture is not fully disclosed, but the behavior and documentation are consistent with hybrid retrieval over internal files plus web search, with the LLM grounding answers in both sets of sources and providing citations.[11][47] [^5]

Comparative Takeaways

Across these products, the most engaged features are document- or workspace-scoped Q&A ("ask my docs"), summarization, and synthesis workflows (reports, outlines, flashcards), all of which rely on RAG-style retrieval plus LLM generation over user content. Leading systems differ most in (1) how clearly they show citations and provenance, (2) how well they support large, evolving knowledge bases, and (3) how deeply they integrate into a PKM or workflow system versus acting as a thin layer over files. The strongest differentiators for a new product are likely to come from richer graph-based sense-making, fine-grained control over retrieval scope (e.g., selected Zettels), and robust, transparent evaluation and hallucination mitigation rather than from commodity components like embeddings and vector stores alone.[2][3] [4][39] [7][36] [13][33] [15][16]

RAG Pipeline Components for Zettel-Based Personal Knowledge

Chunking Strategies

Chunking strategies for RAG include fixed-size token windows with overlap, structure-aware or document-aware chunking (using sections, headings, or paragraphs), semantic chunking based on topic boundaries, and "late chunking" where chunks are defined after retrieval or reasoning. For general text, guidance from vector DB vendors and RAG practitioners suggests starting with fixed-size chunks in the 256–1024 token range with 10–20 percent overlap to avoid losing context at boundaries, using sentence or paragraph splits to avoid mid-sentence cuts. For short-form content like tweets or Reddit comments where the entire text is often < 280–500 characters, the Zettel or post itself can be treated as a single chunk and sometimes combined with its summary and tags, while for long-form video transcripts,

sliding-window or dialogue-turn-aware chunking is needed to keep semantic coherence and capture temporal context.[48][49] [50][31]

Document-aware chunking for structured documents (e.g., section-based splitting of articles and transcripts) has been shown in domain-specific RAG systems, such as legal RAG for EU electricity market regulation, to improve retrieval precision and context focus by respecting article and structural boundaries and combining this with sliding windows. Late chunking, such as extracting a focused subgraph or summarizing communities before constructing the final context, appears in graph-based RAG frameworks like GraphRAG, where the system first builds a knowledge graph and community structure, then generates community summaries as the actual "chunks" used for retrieval and prompting. Choice of chunking remains an open research area, especially around optimal sizes for different domains and the trade-off between information density and retrieval granularity; production systems typically explore multiple strategies empirically.[51][21] [26][48]

Embedding Models for Heterogeneous Personal Knowledge

Embedding models are central to dense retrieval; the Massive Text Embedding Benchmark (MTEB) and similar leaderboards track embedding performance across retrieval, clustering, reranking, and other tasks. Recent leaderboards show that models such as NVIDIA's NV-Embed-v2 and Llama-Embed-Nemotron-8B lead in overall and retrieval-specific scores, with NV-Embed-v2 achieving an average score of 72.31 across 56 tasks and a retrieval subcategory score of 62.65. Open-weight models such as BGE-large-en-v1.5 achieve MTEB scores in the mid-60s (e.g., 64.23), focusing on high-quality English semantic search, and are widely used in RAG systems for document retrieval, similarity search, and clustering.[17][28] [29][52]

Cloud-hosted models like OpenAI's text-embedding-3-small prioritize cost-effectiveness, priced at 0.02 USD per million tokens and designed as a faster, more performant replacement for older ada embeddings, making them attractive when tight integration with OpenAI-hosted LLMs or low operational overhead is desired. Leaderboard maintainers and practitioners highlight that open-weight models such as NV-Embed-v2 and BGE variants are production-ready, especially when self-hosting is acceptable, and that choosing between them depends on language coverage, latency constraints, and integration with existing infrastructure. For heterogeneous personal knowledge containing tweets, summaries, tags, and long transcripts, models that perform strongly on retrieval and semantic similarity tasks (rather than niche tasks) are preferred, but the exact best model remains somewhat domain-dependent and is best validated via task-specific evaluation.[53][28] [29][17]

Vector Stores and Supabase pgvector

Vector stores for RAG must support efficient nearest-neighbor search, metadata filtering, and often multi-tenant isolation; options include dedicated vector DBs (e.g., Milvus, Pinecone, Qdrant) and Postgres extensions such as pgvector. Supabase offers managed pgvector with support for HNSW and IVFFlat indexes, distance operators for Euclidean, inner-product, and cosine similarity, and maximum dimensions of 2,000 for vector and higher for halfvec and bit types, making it suitable for modern embedding models up to typical dimensionalities. Supabase guidance recommends HNSW indexes for performance and robustness under

changing data, and benchmarks show that pgvector can handle production workloads with around one million OpenAI embeddings at approximately 1,800 queries per second with 91 percent accuracy or 670 queries per second with 98 percent accuracy on suitable hardware and index configurations.[18][30] [^31]

For multi-tenancy, Supabase suggests three approaches: separate databases per tenant, separate schemas per tenant, or shared schema with row-level security (RLS); community discussions note that using a shared schema with a tenant_id column and RLS is the most straightforward and scalable approach in many SaaS architectures. Neon and others have demonstrated per-tenant Postgres-with-pgvector architectures where each user or workspace gets a dedicated database, which improves isolation at the cost of more connections and operational complexity. For a product already on Supabase/Postgres, continuing with pgvector for per-user or per-workspace indexes is generally well supported and avoids the cognitive and operational overhead of introducing a separate vector database, though extremely large-scale deployments may eventually justify specialized vector services.[54][55] [^56]

Retrieval Methods: Dense, Sparse, and Hybrid

Dense retrieval uses embedding-based similarity search (e.g., cosine similarity over pgvector) and excels at semantic matching, especially when user queries differ lexically from source documents. Sparse retrieval methods such as BM25, implemented via classical inverted indexes, remain highly effective for keyword-sensitive domains where exact matches (IDs, names, error codes) are important and are valued for speed, interpretability, and low cost; practitioners note that BM25 is often hard to beat on precision and latency when exact tokens matter. Hybrid retrieval, combining BM25 and dense embeddings, tends to outperform either method alone on benchmarks like BEIR, with empirical examples showing improved mean reciprocal rank (MRR) when fusing BM25 and dense results compared to each individually (e.g., MRR@10 improving from 67 percent BM25-only and 71 percent dense-only to 82 percent hybrid in one reported experiment).[57][31]

For short texts such as tweets and Reddit posts, sparse methods can be competitive because queries and documents share vocabulary and length, while dense embeddings help with paraphrased or conceptually related queries that do not share tokens. In production RAG, hybrid retrieval is increasingly recommended as the default to balance robustness, precision, and recall across diverse query types, although the optimal fusion strategy (e.g., weighted sum of normalized scores, rank fusion) and k-values remain open design choices to be tuned per domain.[21][31] [^57]

Reranking: Cross-Encoder Rerankers

Rerankers are cross-encoder models that take a query and a candidate document (or passage) and compute a relevance score, enabling a second-stage ranking over a small candidate set retrieved by BM25 or dense search. Documentation for BGE-Reranker explains that cross-encoders such as bge-reranker-base and bge-reranker-large are trained on high-quality English and Chinese data and achieve state-of-the-art performance for their size, typically used to rerank the top ~100 documents and then select the top few for context.

Benchmarks from reranker vendors show that reranking significantly boosts metrics such as NDCG@10; for example, ZeroEntropy reports that its zerank-1 cross-encoder achieves higher NDCG@10 and lower latency than Cohere Rerank 3.5 on tested workloads while Cohere rerankers are widely used as a baseline, indicating the general value of cross-encoder reranking for top-k quality.[58][19]

The trade-off is latency and cost, since cross-encoders are heavier than bi-encoder embeddings; however, for candidate lists of moderate size (e.g., k ≤ 100), the additional 100–300 ms of latency per query is often acceptable for chat use cases when it materially improves relevance. Exact model choice depends on hosting (self-hosted vs API), language coverage, and performance requirements, and remains an area of active experimentation, but the principle of two-stage retrieval plus reranking is well supported by benchmarks and practitioner reports.[19][21] [^58]

Query Transformation: HyDE, Multi-Query Expansion, Decomposition

Query transformation methods generate alternative or enriched forms of the user's query to improve retrieval; examples include hypothetical document embeddings (HyDE), multi-query expansion, and query decomposition. The HyDE method generates a hypothetical relevant document using an instruction-following LLM, then embeds that document and uses it to retrieve similar real documents; experiments in the original HyDE paper report that this approach significantly outperforms unsupervised dense retrievers like Contriever and approaches the performance of fine-tuned retrievers across web search, QA, and fact verification tasks. LangChain's implementation exposes HyDE as a retriever that generates hypothetical documents from queries before embedding and retrieving, and the library also supports multi-query retrievers that use an LLM to generate multiple reformulations of a query and union their retrieved documents.[59][60] [61][62]

OpenAI and LangChain materials highlight that query expansion and HyDE can be particularly helpful when queries are under-specified, when the corpus is noisy, or when vocabulary mismatch is common, but they add latency and cost due to multiple LLM calls and retrieval passes. Query decomposition, where complex questions are broken into subquestions whose answers are then combined, is another emerging pattern, especially for multi-step reasoning or long-horizon tasks, but its benefits depend heavily on the complexity of queries and the ability to reliably detect subquestions. Overall, these methods are promising but should be evaluated empirically for the specific Zettel corpus; they are most valuable for complex or ambiguous queries rather than everyday fact lookup.[21][61]

Knowledge Graph Integration and GraphRAG / KG-RAG

Using knowledge graphs alongside vector retrieval is a natural fit for personal knowledge systems; frameworks such as GraphRAG and knowledge-graph-based RAG provide concrete patterns. Microsoft's GraphRAG approach constructs a knowledge graph from the corpus using an LLM, builds a community hierarchy over the graph, generates community summaries, and then uses these graph structures and summaries to augment prompts at query time, showing substantial improvement in answering narrative and high-level questions compared to plain semantic-search RAG. Official GraphRAG documentation describes a process of

building a NetworkX graph, running hierarchical community detection (e.g., Leiden), and generating community summaries via an LLM, which then serve as higher-level semantic units for retrieval.[51][63] [26][27]

LlamaIndex provides a Knowledge Graph RAG Query Engine that supports Graph RAG (subgraph extraction around key entities) and NL2GraphQuery (generating structured graph queries from natural language), combining graph-based subgraph retrieval with unstructured text retrieval. Research on hybrid structured-unstructured retrieval methods such as SKETCH shows that combining semantic text retrieval with knowledge graphs can improve RAG performance on metrics like answer relevancy and context precision compared to text-only baselines, demonstrating the value of graph-augmented retrieval. However, how best to combine graph-based scores with embedding similarity and how to design scalable graph schemas for heterogeneous personal data remain open research questions, and production systems often adopt pragmatic, heuristic combinations.[64][27]

Context Assembly and Ranking

Context assembly involves selecting, ordering, and formatting retrieved chunks for inclusion in the LLM prompt; it must balance information coverage, relevance, and redundancy within the LLM's context window. GraphRAG's approach uses community-level summaries and PageRank-style community scoring over the knowledge graph to select relevant communities and then their associated passages, effectively ranking context by graph-based centrality and relevance to the query. SKETCH similarly leverages structured knowledge to maintain context integrity and improve context precision by integrating knowledge-graph-derived signals into retrieval and context selection.[64][63] [26][31]

Beyond graph-specific techniques, RAG evaluation literature emphasizes that context selection should maximize token-level relevance while minimizing extraneous content; frameworks like RAGBench, ARES, and RAGChecker propose token-aware or span-aware metrics that quantify the trade-off between context density and recall, suggesting the value of ranking strategies that prioritize highly relevant spans rather than entire long passages. Concrete strategies include: scoring chunks by a weighted combination of embedding similarity, BM25 score, reranker score, and graph-based centrality; merging or trimming overlapping chunks; and grouping context by Zettel or source to preserve provenance, all of which must be tuned empirically but are supported by evaluation frameworks.[65][22] [66][67]

Generation Layer: Prompting and Hallucination Mitigation

The generation layer uses an LLM to answer the user's question given the selected context and is responsible for grounding, citations, and answering style. RAGAS and hallucination survey papers emphasize that hallucination is an inherent limitation of LLMs that can be reduced but not eliminated, and that RAG mitigates hallucinations by constraining answers to retrieved context and enabling explicit faithfulness metrics, but does not guarantee hallucination-free behavior. Legal and domain-specific RAG systems show that designing prompts that explicitly instruct the model to answer only from context, state when information is missing, and cite sources improves faithfulness, but hallucinations can still occur, especially when retrieval misses or context is ambiguous.[48][20] [21][68] [69][31] [^70]

Hallucination surveys and benchmarks explore mitigation strategies such as post-hoc hallucination detection models (e.g., HHEM-2.1-Open within RAGAS), human-in-the-loop review for high-stakes domains, and self-consistency checks across multiple generations, but these introduce cost and complexity and are not foolproof. For a personal-knowledge chatbot, practical mitigations include: clearly separating quoted context from model commentary, always showing which Zettels support each claim, encouraging the model to admit uncertainty, and providing user controls to inspect underlying Zettels, mirroring practices in systems like NotebookLM and Perplexity that display inline citations and allow users to inspect sources.[5][71] [21][13] [14][15] [72][73]

Evaluation: Retrieval and Generation Quality

Evaluating RAG involves both retrieval quality and generation quality; RAGAS, RAGBench, ARES, RAGChecker, and survey papers propose frameworks and metrics for this. RAGAS introduces metrics such as faithfulness (measuring how consistent the answer is with retrieved context), context precision (fraction of retrieved context that is relevant), context recall (fraction of ground-truth relevant content retrieved), and answer relevancy (alignment with the reference answer), computed without necessarily requiring ground-truth relevance labels by leveraging LLM-based judges. RAGBench provides an explainable benchmark with labeled examples from industry corpora and the TRACe evaluation framework, which decomposes evaluation into context retrieval, answer grounding, and response quality, enabling actionable feedback for optimizing RAG systems.[71][20] [21][22] [66][67] [^74]

TruLens is an open-source framework for evaluating and monitoring RAG systems that integrates with LangChain and LlamaIndex, offering metrics like context relevance and groundedness (faithfulness) via LLM-based feedback functions and dashboards to visualize and compare pipeline versions. Broader evaluation surveys recommend measuring retrieval with information-retrieval metrics such as recall, precision, mean reciprocal rank (MRR), and NDCG, and generation with faithfulness, answer relevance, and user-centric measures (task success, satisfaction), often using synthetic or curated QA benchmarks over the product's own corpus. Evaluation remains an active research area; automated LLM judges offer speed but can introduce bias, so combining them with targeted human evaluation is considered best practice.[65][75] [21][24] [^25]

End-to-End Architecture Recommendation for the Zettel-Based System

High-Level Architecture Overview

For a FastAPI + Supabase-based webapp where users maintain Zettels (YouTube videos, tweets, Reddit posts) already summarized, tagged, and connected in a Knowledge Graph, the recommended architecture is a hybrid Graph- and vector-augmented RAG pipeline with per-user isolated indexes and multi-stage retrieval and ranking. The core components are:[18][55] [26][27]

- Data layer: Supabase/PostgreSQL with pgvector for embeddings, relational tables for Zettels and their metadata, and graph tables (edges) representing the knowledge graph.

- Indexing layer: per-user (or per-workspace) vector and text indexes over Zettels, summaries, and relevant graph-derived text (e.g., community summaries), using HNSW or IVFFlat for pgvector and Postgres full-text indexes for BM25-like retrieval.[30][18]
- Retrieval layer: hybrid dense + BM25 retrieval over the selected Zettels, optional graph-based expansion via GraphRAG-style community subgraphs, and cross-encoder reranking (e.g., BGE-Reranker) to refine top-k candidates.[57][19] [^26]
- Context assembly layer: merging candidate chunks and graph-derived summaries into a constrained context window ordered by a composite relevance score, preserving Zettel boundaries and citations.
- Generation layer: an LLM (e.g., GPT-4-class or Gemini/Groq equivalent) orchestrated via FastAPI, with prompts that enforce grounding, citations, and explicit references to Zettel IDs.
- Evaluation and monitoring: RAGAS and TruLens integrated into the pipeline to measure faithfulness, context precision, and answer relevance, and to iterate on retrieval and prompt settings.[20][24] [74][25]

This architecture keeps all storage and indexing inside Supabase, avoiding the need for an external vector DB while leveraging pgvector's HNSW performance and RLS-based multi-tenancy. It treats the existing Knowledge Graph as a first-class asset by using graph-based selection and community summaries inspired by GraphRAG and LlamaIndex's knowledge-graph query engines, enabling richer retrieval than plain semantic search.[63][55] [26][27] [18][30]

Data Model and Indexing Design

In Supabase/PostgreSQL, each user has Zettels stored in a `zettels` table with columns such as `id`, `user_id`, `source_type` (youtube, tweet, reddit), `raw_text` (or transcript), `summary`, `tags` (array), `created_at`, and `selected` (for user selection scopes). The knowledge graph is represented by tables such as `kg_nodes` (node_id, user_id, zettel_id, type, label) and `kg_edges` (edge_id, user_id, source_node_id, target_node_id, relation_type), allowing graph queries via joins or graph extensions. Embeddings are stored as pgvector columns on `zettels` and possibly on `kg_nodes` or community summaries, with a per-user index either via a shared table using `user_id` plus RLS or per-user schema if stricter isolation is required.[18][55] [26][27]

Vector indexes use pgvector with HNSW or IVFFlat; Supabase recommends HNSW for better performance and robustness, and supports cosine distance, which aligns well with most normalized embeddings. Full-text search for BM25-style retrieval is provided by Postgres's `tsvector` and GIN indexes, allowing hybrid retrieval by combining BM25 scores with vector similarities. For multi-tenancy, the shared-table-with-RLS approach reduces connection overhead relative to per-database tenant models, while still enforcing per-user isolation through row-level policies as recommended in Supabase's multi-tenancy discussion.[55][56] [31][18]

Embedding and Chunking Strategy for Zettels

Each Zettel is treated as a logical unit; for short content (tweets, short Reddit posts), the chunk is the Zettel itself, optionally concatenating the user-generated summary and tags to enrich the semantic signal. For longer content such as YouTube transcripts, the system stores both the full transcript and pre-chunked segments (e.g., 256–512 tokens with 10–20 percent overlap) linked to the parent Zettel via `parent_zettel_id`, allowing more granular retrieval without losing Zettel-level context. Summaries of long Zettels are also embedded and stored as separate vectors, enabling retrieval at both coarse (summary) and fine (chunk) levels.[49][50]

Embedding models should be chosen based on operational constraints; for a cloud-hosted system comfortable with external APIs, a model like OpenAI's text-embedding-3-small offers strong performance at low cost and tight integration with major LLM APIs. For teams preferring self-hosting or higher retrieval quality, open-weight models such as NV-Embed-v2 or BGE-large-en-v1.5, which lead or perform strongly on MTEB retrieval tasks, can be deployed behind an internal service and integrated with the FastAPI backend. Given the heterogeneous nature of the content (short and long texts, tags), models strong on retrieval and semantic similarity across general English text are appropriate, and the decision between cloud vs self-hosted should be validated via RAGAS or custom retrieval benchmarks over a sample of users' Zettels.[17][53] [28][29]

Retrieval and Graph-Augmented Retrieval Flow

When a user opens the chatbot and selects a subset of Zettels, the system records the selected Zettel IDs and uses them to restrict retrieval to that subset by filtering on `zettels.id` in both vector and BM25 queries. The primary retrieval stage runs hybrid retrieval: [^31]

- Dense: a pgvector search over the selected Zettels' embeddings (and, for long Zettels, their chunk embeddings) using cosine distance, returning top-N candidates.
- Sparse: a Postgres full-text search (BM25-like) over `raw_text`, `summary`, and tags for the same selection, returning top-M candidates.[18][31]

The results are fused using a simple rank- or score-based fusion method (e.g., normalized scores weighted 0.5 each) as recommended in hybrid retrieval practice, exploiting BM25's strength on exact term matches and dense retrieval's semantic coverage. From this candidate pool, the system optionally performs graph-augmented expansion: for each candidate Zettel, it traverses the knowledge graph to collect neighboring nodes (linked Zettels or entities) up to a limited depth and uses precomputed community assignments or graph centrality scores to prioritize nodes likely to be relevant, following patterns from GraphRAG and knowledge-graph RAG engines.[57][63] [26][27]

Graph-aware retrieval may also incorporate community summaries: offline, the system periodically runs community detection on each user's knowledge graph (e.g., Leiden algorithm via NetworkX) and generates LLM-based summaries of each community, storing them as pseudo-Zettels with embeddings. At query time, the system searches over these community summaries alongside individual Zettels; if a community summary ranks highly, its

constituent Zettels receive a relevance boost, effectively implementing a PageRank-style relevance propagation over the graph as in GraphRAG. This enables better handling of broad or high-level questions that span multiple Zettels.[64][63] [^26]

Reranking and Context Assembly

After hybrid and graph-augmented retrieval, a candidate set of, for example, 50–100 Zettel chunks or summaries is reranked by a cross-encoder reranker such as BGE-Reranker-base or a hosted rerank API, which produces a scalar relevance score per query–document pair. Only the top K (e.g., 10–20) are selected for context, with chunks from the same Zettel grouped together and truncated as needed to fit the LLM's context window, preserving complete sentences and including Zettel metadata (title, tags, source) for grounding.[58][19] [49][31]

For each context chunk, the system includes an explicit identifier (e.g., `Zettel #123`, `YouTube: [title] @ timestamp`) so that the LLM can cite them in its answer; this mirrors NotebookLM's and Perplexity's use of inline citations referencing passages and improves transparency. The final context ordering is determined by the reranker score possibly combined with graph-based centrality or community relevance, ensuring that highly central and highly relevant chunks appear earlier, which is important given LLMs' tendency to focus on earlier context.[5][64] [26][13] [14][15]

Generation Layer and Prompting

The generation layer is implemented as a FastAPI endpoint that orchestrates LLM calls, passing the assembled context, the user query, and system-level instructions. The prompt should:

- Instruct the LLM to answer using only the provided context and to explicitly say when information is not available.
- Require that key factual claims be associated with Zettel citations (e.g., "Based on Zettel #123 and #145, …").
- Encourage structured answers (headings, bullet lists) when appropriate, but avoid making up references or Zettels not present in context.

RAG-focused evaluation and hallucination studies suggest that such explicit instructions, combined with source-cited context, improve faithfulness, although hallucinations cannot be fully eliminated. For high-stakes or power-user features (e.g., publishing a summary to a knowledge base), additional safeguards such as running RAGAS faithfulness checks or hallucination detectors on the generated answer before displaying or storing it can be employed. Choice of LLM can be flexible (OpenAI, Gemini, Anthropic, open-weight models via Ollama), but given the need for strong reasoning over heterogeneous content, a GPT-4-class or equivalent model is recommended.[71][20] [21][69] [^70]

Evaluation and Iteration Loop

To ensure that the RAG pipeline remains accurate and trustworthy, the system should log interactions (queries, retrieved contexts, answers, user feedback) and periodically evaluate performance using RAGAS and TruLens. RAGAS can compute metrics such as faithfulness, context precision, context recall, and answer relevancy over a curated or synthetic QA set built from the Zettels, enabling comparison of different retrieval and chunking configurations. TruLens can wrap the end-to-end pipeline and record metrics such as context relevance and groundedness for live traffic, visualizing where retrieval or generation falls short and guiding improvements.[71][20] [24][74] [^25]

Survey work and RAGBench suggest adopting a unified evaluation process that separately measures retrieval and generation quality, using metrics like recall, precision, MRR, NDCG for retrieval and faithfulness and answer relevance for generation, and that synthetic benchmarks tailored to the product's domain provide useful, though imperfect, proxies for real-world performance. Combining automated metrics with targeted human reviews of sampled conversations—particularly for failure cases reported by users—offers the most robust feedback loop.[65][21] [22][67]

Stack and Library Recommendations

Given the existing stack and the desire to retain control over architecture, a thin orchestration layer over core components is preferable to fully abstracted frameworks. LangChain is well-suited for composing document loaders, splitters, retrievers (including hybrid and HyDE/MultiQuery), rerankers, and LLM calls, and it includes integrations for pgvector and Postgres-based retrieval. LlamaIndex offers powerful index and query-engine abstractions, including knowledge-graph RAG and GraphRAG-style community building, and has integrations with Postgres/pgvecto.rs, making it a strong candidate for the graph-enhanced part of the pipeline.[61][27] [31][76]

A pragmatic choice is to use:

- LangChain for embedding, hybrid retrieval, and RAG chain composition (including HyDE or multi-query retrievers where needed).[31][61]
- LlamaIndex's graph-related components (e.g., Knowledge Graph RAG Query Engine or GraphRAG integration) to build and query the user's knowledge graph, with data stored in Supabase and accessed via connectors.[63][27]
- Direct SQL or SQLAlchemy for Supabase/Postgres integration when simple vector or text retrieval is sufficient, to avoid over-abstraction.

This combination allows fine-grained control and leverages best-of-breed graph-RAG tools without locking the system into a monolithic framework.

Hardest Engineering Challenges and How Others Address Them

Key engineering challenges include:

1. **Retrieval quality and coverage across heterogeneous content.** Ensuring that both short and long Zettels are retrieved appropriately is non-trivial; hybrid retrieval plus carefully tuned chunking and graph-based expansion are required, and even then, retrieval misses can cause hallucinations or incomplete answers. Systems like NotebookLM, Notion Q&A, and legal RAG applications address this by combining multiple retrieval strategies, constraining scopes (e.g., notebooks, workspaces), and iteratively tuning chunk sizes and index parameters.[39][57] [64][48] [49][26] [^15]
2. **Managing hallucinations and user trust.** As research on hallucinations emphasizes, hallucination is inevitable in LLMs and cannot be fully eliminated, even with RAG. Production systems mitigate this by grounding answers in explicit citations (NotebookLM, Perplexity), enforcing strict prompts to "answer only from sources," adding user-facing warnings, and, in higher-stakes domains, incorporating automated hallucination detectors or human review.[5][71] [13][15] [68][73] [69][70]
3. **Scalable multi-tenancy and performance.** Storing embeddings and graph data for many users can stress Postgres connections and memory; Supabase and Neon guidance indicates that shared schemas with tenant_id and RLS are scalable, while per-tenant databases require careful connection pool management. pgvector performance benchmarks show that with proper HNSW indexes and hardware, millions of vectors can be queried at hundreds to thousands of QPS, but tuning (index parameters, warm-up, concurrency) is crucial.[54][18] [55][30] [^56]
4. **Graph construction and evolution.** Maintaining a high-quality knowledge graph as users add, delete, and update Zettels is challenging; GraphRAG and hybrid methods like SKETCH show that graph construction and community building can be done offline, but keeping them fresh and consistent requires background jobs and careful schema design. Incremental graph updates, periodic community recomputation, and versioning of graph states are common approaches.[64][63] [^26]
5. **Robust evaluation and continuous improvement.** Evaluation frameworks such as RAGAS, RAGBench, ARES, RAGChecker, and TruLens provide tools, but integrating them into CI/CD and using their metrics to drive changes demands discipline and infrastructure. Production teams address this by defining target metric thresholds (e.g., minimum faithfulness) and running regression tests on curated QA sets whenever retrieval or prompts are changed, complemented by dashboards and alerting for live traffic.[20][22] [66][67] [^25]

Addressing these challenges requires iterative experimentation, careful logging, and a willingness to treat the RAG system as an evolving product rather than a static pipeline, but existing products and research provide strong patterns and tools for doing so.

References

1. Vault AI Chat with RAG, file deletion/creation, AI generated content ... - Vault AI Chat - Obsidian Plugin An AI-powered chat assistant that lives inside your Obsidian vault. ...
2. How to Use Mem with Your Team: Collective Intelligence - Enter Mem, the AI notes app revolutionizing how teams collaborate and organize their collective know...
3. Introducing Q&A: get instant answers to your questions in ... - It's a way for you to get instant answers to any question using information across your knowledge, d...
4. Reader: Frequently Asked Questions | annotated by Scott - Changelog October 27, 2023 Updated instructions for adding document notes to include mobile and brow...
5. What is Internal Knowledge Search | Perplexity Help Center - Internal Knowledge Search allows both Perplexity Pro and Enterprise users to search across internal ...
6. Mem AI: Smart Notes and Workspace for Teams and Pros - Mem AI is an AI-powered notes app and collaborative workspace that helps individuals and teams captu...
7. Anyone using Google's NotebookLM as a PKM? - Anyone using Google's NotebookLM as a PKM?
8. Exploring Notion AI Q&A: Your New Workspace Companion - Notion AI Q&A can only answer questions based on the content each specific user has access to. It re...
9. Why NotebookLM is NOT a Personal Knowledge Management (PKM) System - Google's NotebookLM is a powerful tool, but is it a true Personal Knowledge Management (PKM) system?...
10. Lector: Preguntas frecuentes - Pro Tip: Use Ghostreader on this doc One of the coolest features of Reader is an AI reading assist...
11. What are Spaces? | Perplexity Help Center - Spaces are dedicated workspaces that allow individual users and teams to organize, collaborate, and ...
12. Add or discover new sources for your notebook - Computer - Important: At this time, the NotebookLM mobile app may have limitations to this feature. Learn more ...
13. NotebookLM: This AI Is Grounded in Your Documents, Not the ... - Drowning in research? NotebookLM is your private AI thinking partner. Upload your files, generate re...
14. How to Use Internal Knowledge in Perplexity | PDF - Perplexity's Internal Knowledge Search allows Pro and Enterprise Pro users to efficiently search and...
15. NotebookLM: What It Is, What It's Actually Good For, and When You ... - A clear, honest guide to Google NotebookLM: how it works, who it’s for, how it differs from ChatGPT ...
16. NotebookLM Limitations: 8 Gaps Google Won't Tell You - NotebookLM has eight limitations that matter for serious research: source caps per notebook, isolate...
17. Embedding Model Leaderboard: MTEB Rankings March 2026 - Rankings of the best embedding models by MTEB scores, comparing retrieval quality, dimensions, speed...
18. Supabase: .../content/guides/ai/vector-indexes.mdx | ...
19. BGE-Reranker — BGE documentation

20. RAGAS: Automated Evaluation of Retrieval Augmented Generation - ...Evaluating RAG architectures is, however, challenging because there are several dimensions to con...
21. Evaluation of Retrieval-Augmented Generation: A Survey - ...To better understand these challenges, we conduct A Unified Evaluation Process of RAG (Auepora) a...
22. RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems - .... RAGBench examples are sourced from industry corpora such as user manuals, making it particularl...
23. Context Precision - Context Precision is a metric that evaluates whether all of the ground-truth relevant items present ...
24. Evaluate and Optimize RAG with TruLens (full tutorial) - Systematic evaluation is the key piece in taking your RAG-systems from just a cool demo into somethi...
25. Best Practices for Evaluating RAG Systems - Learn how to effectively implement and evaluate retrieval-augmented generation systems using popular...
26. Welcome - GraphRAG
27. Knowledge Graph RAG Query Engine - LlamaParse - LlamaIndex
28. nvidia/NV-Embed-v2 - NV-Embed is a generalist embedding model encompassing retrieval, reranking, classification, clusteri...
29. BGE Large v1.5 | Products - BGE Large 1.5 is a high-performance English-focused embedding ... MTEB score of 64.23. This model ex...
30. pgvector 0.4.0 performance - Supabase - There's been a lot of talk about pgvector performance lately, so we took some datasets and pushed pg...
31. What is RAG (Retrieval-Augmented Generation) - The core RAG components are the document processing pipeline, embedding models, vector database, ret...
32. Application of NotebookLM, a Large Language Model with Retrieval-Augmented Generation, for Lung Cancer Staging - ...due to limitations such as hallucinations and insufficient referencing. To address these issues, ...
33. NotebookLM: A Guide With Practical Examples - DataCampwww.datacamp.com › tutorial › notebooklm - Learn how to make the most of NotebookLM by utilizing its podcast-generating feature and its ability...
34. Mini-RAG - Obsidian Stats - Obsidian Plugin: Mini-RAG - null by John Wheatley. Latest version: 1.0.0 released on 10 months ago
35. Mini-RAG - Obsidian文档咖啡豆版 - 利用本地运行的 LLM 或 AI，为您的笔记提供检索增强生成 (RAG)。
36. How to Use Mem Chat: 15 Queries That Will Change How You ... - A practical guide to Mem Chat with 15 real queries organized by use case: weekly reviews, meeting pr...

37. 11 Best AI Second Brain Tools 2026 (Notes to Action) - Mem can generate drafts and summaries based on your notes but lacks automations, agents, or integrat...
38. A Guide to Building a Second Brain in Mem - A guest post by Srini Rao
39. The ultimate guide to Notion AI Q&A in a knowledge hub - Explore the features, limitations, and real-world performance of Notion AI Q&A for your knowledge hu...
40. Get answers about work content faster with Q&A - Learn how Q&A can answer questions to help you make sense of the information in your workspace, and ...
41. How Notion is tackling one of AI's biggest looming problems - Retrieval augmented generation (or RAG) seems to be increasingly a go-to solution for trying to comb...
42. 9 ways to use Artificial Intelligence in Readwise Reader to prepare for your next assignment — techforword - Artificial Intelligence tools – especially ChatGPT – are taking the world by storm. How can translat...
43. What is Ghostreader? - Readwise Docs - Ghostreader comes with a few default prompts that enable you to do things like quickly summarize the...
44. Ghostreader in Readwise Reader - Outliner Software
45. Reader (by Readwise) is Goated - I am impressed with the Readwise team for creating a great app with a solid AI integration that prov...
46. What are Perplexity Spaces? - Business Libraryanswers.businesslibrary.uflib.ufl.edu › genai › faq
47. Perplexity Spaces vs. Zine: Web Search is Great, But What About ... - Perplexity Spaces combines your uploads with web search, but teams need secure, private search acros...
48. Engineering Trustworthy Retrieval-Augmented Generation for EU Electricity Market Regulation - The regulatory framework governing EU electricity markets is highly complex, fragmented across multi...
49. How do I implement efficient document chunking for RAG ... - First, choose a chunking method based on your data type. For general text, fixed-size chunks (e.g., ...
50. Using Gen AI to Query Video Transcripts - Fetches a YouTube video's transcript · Splits it into manageable chunks · Embeds those chunks using ...
51. GraphRAG: Unlocking LLM discovery on narrative private ... - Microsoft Research's new approach, GraphRAG, uses the LLM to create a knowledge graph based on the p...
52. Introducing MTEB v2: Evaluation of embedding and ... - Introducing MTEB v2: Evaluation of embedding and retrieval systems for more than just text. Communit...
53. Text Embedding 3 Small - OpenAI Platform - Explore resources, tutorials, API docs, and dynamic examples to get the most out of OpenAI's develop...
54. AI app architecture: vector database per tenant - This repo contains an example of a scalable architecture for AI-powered applications. On the surface...
55. Multi-tenant · supabase · Discussion #1615 - is there a way to create a database with the possibility of Multi-tenant ?

56. Postgres multi-tenant architecture for AI apps (database ... - I have a proposal for an architecture that will make building AI apps that are powered by postgres m...
57. BM25 vs Dense Embeddings: Choosing the Right Retrieval ... - BM25 is fast, interpretable, and brutally effective when keywords matter. It's hard to beat on preci...
58. Latency Benchmark: Cohere rerank 3.5 vs. ZeroEntropy zerank-1 - zerank-1 delivers the fastest, most accurate reranking for AI search—up to 14× faster than Jina and ...
59. HyDE: Precise Zero-Shot Dense Retrieval without ... - We discuss HyDE: a zero-shot learning technique that combines GPT-3's language understanding with co...
60. HyDE: Zero-Shot Dense Retrieval Method - This paper presents HyDE, an innovative method that leverages language models to generate hypothetic...
61. Applying OpenAI's RAG Strategies - OpenAI reported two methods, which you can try: Query expansion: LangChain's Multi-query retriever a...
62. Precise Zero-Shot Dense Retrieval without Relevance Labels - arXiv - While dense retrieval has been shown effective and efficient across tasks and languages, it remains ...
63. Graphragqueryengine
64. SKETCH: Structured Knowledge Enhanced Text Comprehension for Holistic Retrieval - Retrieval-Augmented Generation (RAG) systems have become pivotal in leveraging vast corpora to gener...
65. Enhancing Domain-Specific Retrieval-Augmented Generation: Synthetic Data Generation and Evaluation using Reasoning Models - ...document-level metrics inadequately capture token-resolution retrieval accuracy
that is critical ...
66. RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation - ...evaluation of RAG systems is
still challenging due to the modular nature of RAG, evaluation of lo...
67. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems - ...systems traditionally relies
on hand annotations for input queries, passages to retrieve, and res...
68. Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools - ...in these tools are prone to
"hallucinate," or make up false information, making their use risky i...
69. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions - ...of LLM and then delve into the factors contributing to
hallucinations. Subsequently, we present a...
70. Why Language Models Hallucinate - by AT Kalai · 2025 · Cited by 285 — Abstract. Like students facing hard exam questions, large langua...
71. Faithfulness - The Faithfulness metric measures how factually consistent a response is with the retrieved context. ...

72. Survey of Hallucination in Natural Language Generation - ...based generation is prone to hallucinate unintended text, which degrades the system performance a...
73. Hallucination is Inevitable: An Innate Limitation of Large Language Models - Hallucination has been widely recognized to be a significant drawback for large language models (LLM...
74. Context Precision - The ContextPrecision metric evaluates whether retrieved contexts are useful for answering a question...
75. Compression, Transduction, and Creation: A Unified Framework for Evaluating Natural Language Generation - ...e.g., summarization), transduction (e.g., text rewriting), and creation (e.g., dialog). _Informat...
76. Building the RAG pipeline with LlamaIndex and pgvecto.rs - In this blog post, we will show you how to build a RAG pipeline with pgvecto.rs and LlamaIndex.